

P3105R2

constexpr

std::uncaught_exceptions()

Published Proposal, 2024-04-04

This version:

<https://eisenwave.github.io/cpp-proposals/constexpr-uncaught-exceptions.html>

Author:

[Jan Schultke](#)

Audience:

SG18, LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

[eisenwave/cpp-proposals](#)

Abstract

This paper marks `std::uncaught_exceptions` and `std::current_exception` `constexpr`.

Table of Contents

1 Revision history

1.1 Changes since R1

1.2 Changes since R0

2 Introduction

3 Motivation

4 Possible implementation

4.1 `constexpr` `uncaught_exceptions`

4.2 `constexpr` `current_exception`

4.3 `constexpr` `exception_ptr`

4.4 Non-trivial implementations

4.5 Impact on ABI

5 Design considerations

5.1 Feature-testing

5.2 `std::rethrow_exception`

6 Proposed wording

7 Acknowledgements

References

Normative References

Informative References

§ 1. Revision history

§ 1.1. Changes since R1

- Add note on [\[P3068R0\]](#) feedback at Tokyo 2024.
- Discuss the use of two feature-testing macros.
- Discuss why `std::rethrow_exception` is not being marked `constexpr`.
- Rename the "Overview" section to "Introduction" in line with WG21 conventions.

§ 1.2. Changes since R0

- Mention [\[P3068R0\]](#).
- Bring up future-proofing in [§ 3 Motivation](#).
- Add missing synopsis edits to [§ 6 Proposed wording](#).
- Minor editorial fixes/improvements.

§ 2. Introduction

I propose to mark the function `std::uncaught_exceptions constexpr`.

Currently, this can be done because there can never be an active exception during constant evaluation.

`std::uncaught_exceptions` would always return zero during constant evaluation. Similarly,

`std::current_exception` would always return a null pointer.

[P2996R1] recommends exceptions as an error handling mechanism for reflections, during constant evaluation. [P3068R0] proposes allowing exception throwing in constant expressions. If exceptions were throwable in constant expressions, marking these functions `constexpr` would simply be part of exception support. **However, these proposals is not a prerequisite to this proposal.**

NOTE: [P3068R0] has been seen by EWG at Tokyo 2024 but was not forwarded to CWG, largely due to the lack of implementation experience.

§ 3. Motivation

The motivation is the same as allowing `try-catch` blocks in `constexpr` functions, a feature added to C++20 thanks to [P1002R1]. Allowing the use of `uncaught_exceptions()` in constant expressions makes metaprogramming easier and eliminates special `if constexpr` cases.

A common use case for `std::uncaught_exceptions` is in the implementation of RAII types which execute a function upon destruction, but only if an exception was (not) thrown in the current scope. This is utilized by `std::scope_success` and `std::scope_failure`; see [N4806].

EXAMPLE 1

In the implementation of a `constexpr` stack, one may write:

```
constexpr value_type pop() {  
    // Only decrease the size of the stack if no exception was thrown du  
    // copy/move construction of the returned object.  
    // This ensures a strong exception guarantee.  
    std::scope_success _{[this] { m_size--; }};  
    return std::move(top());  
}
```

It is reasonable to mark such code `constexpr`, and ideally `std::uncaught_exceptions` would not be an obstacle to this.

Besides the quality-of-life aspect, we want to future-proof code. If the user circumvents `std::uncaught_exceptions` by guarding its use with an `if !constexpr` block, this makes the assumption that exceptions aren't throwable in constant expression. That may be true now, but could change in the future, in which case the user will have to rewrite their code to avoid this assumption.

Furthermore, it makes sense to mark `std::uncaught_exceptions`' sister function, `std::current_exception constexpr`. This is done purely for the purpose of consistency. I am not aware of any concrete example of `std::current_exception`'s lack of `constexpr` being an obstacle.

§ 4. Possible implementation

§ 4.1. `constexpr` `uncaught_exceptions`

```
constexpr int uncaught_exceptions() noexcept {  
    if constexpr {  
        return 0;  
    } else {  
        return __uncaught_exceptions_impl();  
    }  
}
```

It is obviously possible for the user to wrap `std::uncaught_exceptions` like this themselves (e.g. [\[ScopeLite\]](#)), but this is an unnecessary burden.

§ 4.2. `constexpr` `current_exception`

```
constexpr exception_ptr current_exception() noexcept {
    if consteval {
        return exception_ptr(nullptr);
    } else {
        return __current_exception_impl();
    }
}
```

§ 4.3. `constexpr` `exception_ptr`

`std::exception_ptr` would also need to be made a literal type. All major three standard libraries implement `exception_ptr` as a wrapper class for `void*`, which makes this easily possible.

- See [libc++](#), [exception_ptr.h](#).
- See [libstdc++](#), [exception_ptr.h](#).
- See [MSVC STL](#), [exception](#).

Simply mark all special member functions `constexpr` and if necessary, guard their implementation with an `if !consteval` block. It is impossible to create an `exception_ptr` that is not a null pointer during constant evaluations.

§ 4.4. Non-trivial implementations

[\[P2996R1\]](#) suggests allowing `throw` in constant expressions. This would mean that `std::active_exceptions`, `std::current_exception`, and `std::exception_ptr` would no longer have such trivial implementations, and further functions such as `std::make_exception_ptr` may be marked `constexpr`.

The bare minimum compiler support needed for this is:

1. The compiler must track all active exceptions "magically", so that `std::active_exceptions()` returns the correct amount, and

`std::current_exception()` returns the current exception. This needs compiler support because such mutable global state normally doesn't exist in constant expressions.

2. `std::exception_ptr` behaves like a type-erased, reference-counted smart pointer. [P2738R1] has been accepted into C++26, adding `constexpr` cast from `void*`. This makes the implementation of such type-erasure in `constexpr std::exception_ptr` feasible.

§ 4.5. Impact on ABI

Multiple functions, including member functions of `std::exception_ptr` would become inline functions if marked `constexpr`. To remain ABI-compatible with existing software, it is necessary to emit these inline function into the runtime library.

libstdc++ already conditionally does this by marking member functions of `std::exception_ptr` `__attribute__((__used__))`. Therefore:

- libstdc++ is not affected.
- MSVC STL already defines all member function of `exception_ptr` as inline functions, and is also not affected.
- libc++ would need to apply similar compatibility measures as libstdc++.

§ 5. Design considerations

While the proposal is largely an "add `constexpr` proposal", there are still a few debateable aspects, discussed below.

§ 5.1. Feature-testing

The proposal currently proposes feature-testing through:

- A new `__cpp_lib_constexpr_current_exception` for `std::current_exception`.
- An update to the existing `__cpp_lib_uncaught_exceptions` macro.

This is intuitive because the `uncaught_exceptions` macro wouldn't be used for feature-detection of `constexpr std::current_exception`. However, two ways of feature-testing for such a small proposal may be seen as excessive.

NOTE: This design choice is currently awaiting feedback from SG10.

§ 5.2. `std::rethrow_exception`

The proposal does not mark `std::rethrow_exception` `constexpr`. Without the ability to *actually* have exceptions during constant evaluation (as proposed by [P3086R0]), `std::rethrow_exception` would be an "always UB" function. Based on the function's preconditions, it is not possible to rethrow a null `exception_ptr`.

Therefore, `std::rethrow_exception` is of little use during constant evaluation. Existing (common) code of the form

```
if (p) std::rethrow_exception(p);
```

... can already be constant-evaluated.

Marking it `constexpr` could be more easily justified if the proposal also altered the existing semantics to do nothing when rethrowing a null pointer; however, that is arguably undesirable.

After all, such a change would encourage the pattern of omitting the `if (p)` part from the code above, which may harm readability since it suggests that rethrowing always happens, even if it only conditionally happens. Such a change would also be asymmetrical with `throw`, which results in `std::terminate` being called if there is no current exception, not in a no-op.

§ 6. Proposed wording

The proposed changes are relative to the working draft of the standard as of [N4917].

Update subclause 17.3.2 [version.syn], paragraph 2 as follows:

```
#define __cpp_lib_constexpr_current_exception    202401L // freestanding,  
[...]  
#define __cpp_lib_uncaught_exceptions          201411L202401L // freesta
```

Update subclause 17.9.2 [exception.syn] as follows:

```
constexpr int uncaught_exceptions() noexcept;

using exception_ptr = unspecified;

constexpr exception_ptr current_exception() noexcept;
```

Update subclause 17.9.6 [uncaught.exceptions] as follows:

```
constexpr int uncaught_exceptions() noexcept;
```

Update subclause 17.9.7 [propagation], paragraph 2 as follows:

exception_ptr is a literal type([basic.types.general]) which meets the requirements of *Cpp17NullablePointer* (Table 36). All expressions which must be valid for a *Cpp17NullablePointer* are constant expressions for a null value of type exception_ptr.

NOTE: This wording is slightly work-in-progress.

Update subclause 17.9.7 [propagation], `current_exception` as follows:

```
constexpr exception_ptr current_exception() noexcept;
```

§ 7. Acknowledgements

The original idea for this paper and a portion of its content have been adopted from a proposal draft by Morwenn.

§ References

§ Normative References

[N4917]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 5 September 2022.
URL: <https://wg21.link/n4917>

§ Informative References

[N4806]

Thomas Köppe. *Working Draft, C++ Extensions for Library Fundamentals, Version 3*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4806.html>

[P1002R1]

Louis Dionne. *Try-catch blocks in constexpr functions*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1002r1.pdf>

[P2738R1]

Corentin Jabot; David Ledger. *constexpr cast from void*: towards constexpr type-erasure*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2738r1.pdf>

[P2996R1]

Wyatt Childers; et al. *Reflection for C++26*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2996r1.html>

[P3068R0]

Hana Dusíková. *Allowing exception throwing in constant-evaluation*. 11 February 2024. URL: <https://wg21.link/p3068r0>

[P3086R0]

Mingxin Wang. *Proxy: A Pointer-Semantics-Based Polymorphism Library*. 16 January 2024.
URL: <https://wg21.link/p3086r0>

[ScopeLite]

Martin Moene. *uncaught_exceptions() wrapper in scope-lite*. URL: <https://github.com/martin-moene/scope-lite/blob/89b274a106363101ea258cb9555a9c6a47ae2928/include/nonstd/scope.hpp#L586-L597>